



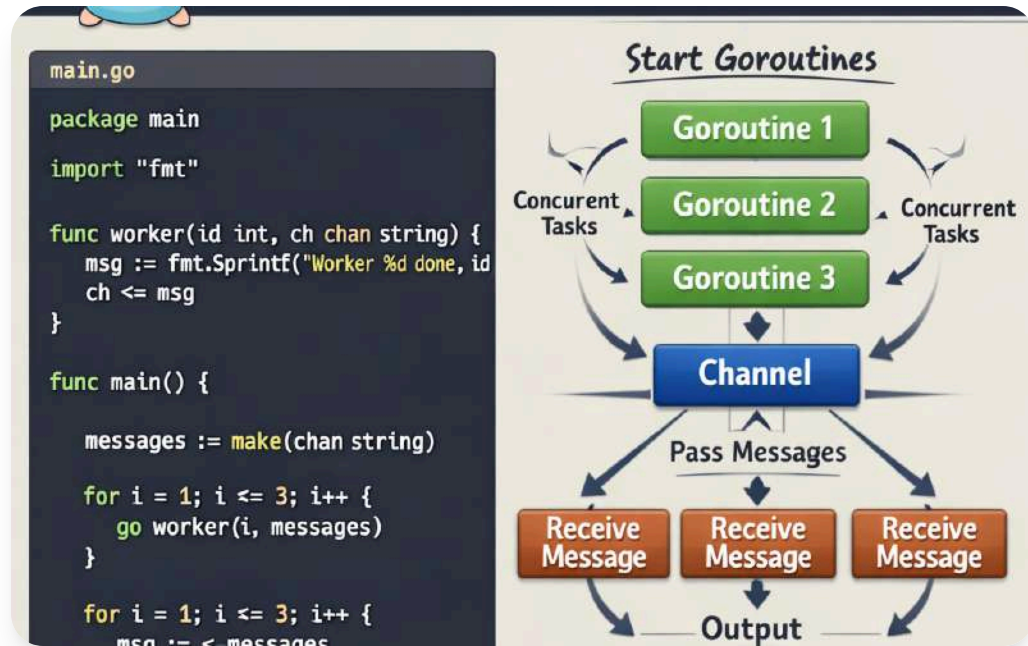
Concurrency dan Goroutines

Materi membahas dasar concurrency di Go, goroutines, sinkronisasi, channel, dan penerapannya pada backend Gin.



Juara Coding

June 2026



Konsep Dasar Concurrency

Concurrency memungkinkan program menangani banyak tugas secara efisien. Di Go, ini dilakukan dengan goroutines dan channels untuk backend modern.

Mengenal Goroutines



Goroutine adalah fungsi concurrent yang ringan dan dijalankan dengan kata kunci *go* di Go.



Sintaks Sederhana

Goroutine dijalankan dengan mudah memakai kata kunci *go* pada sebuah fungsi.



Lebih Ringan

Dibanding thread tradisional, goroutine memakai resource lebih kecil untuk tugas concurrent.



Use Case Backend

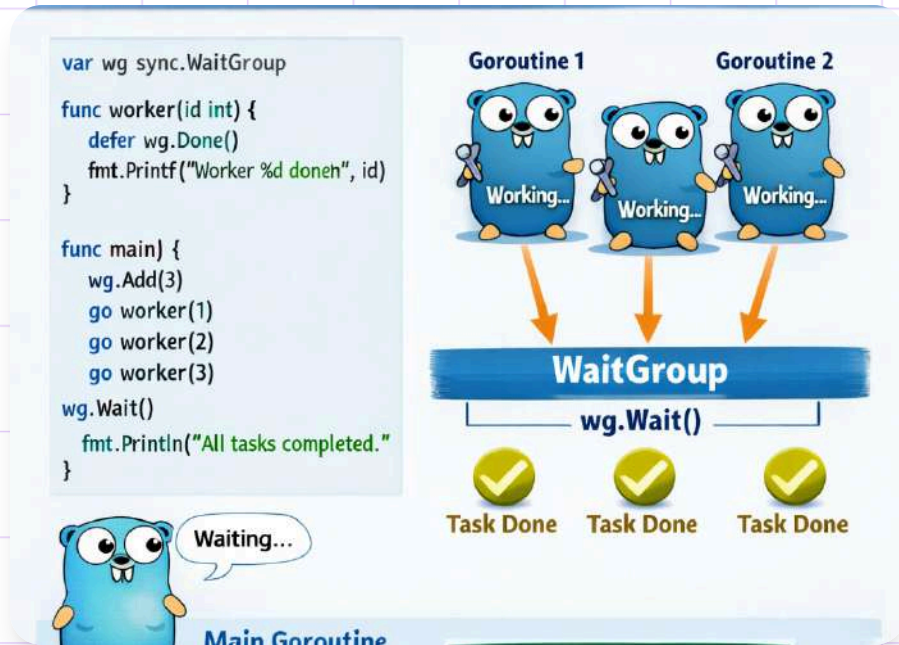
Cocok untuk logging asynchronous, notifikasi, dan pengambilan data paralel.

```
4 import "fmt"
5
6 func main() {
7     go printNumbers()
8     go printLetters()
9     fmt.Println('Main function done')
10 }
11
12 func printNumbers() {
13     for i := 1; i <= 5; i++ {
14         fmt.Printf('Number: %d', i)
15         time.Sleep(500 * time.Millisecond)
16     }
17 }
18
19 func printLetters() {
20     for c := 'A'; c <= 'E'; c++ {
21         fmt.Printf('Letter: %cn', c)
22         time.Sleep(400 * time.Millisecond)
23     }
24 }
```

Dasar Penggunaan Goroutine

Contoh ini menunjukkan fungsi berjalan di goroutine terpisah. Tanpa sinkronisasi, program bisa selesai sebelum goroutine menuntaskan pekerjaannya.

Sinkronisasi dengan WaitGroup



WaitGroup pada package sync menunggu beberapa goroutine selesai sebelum proses utama berlanjut.



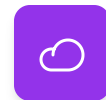
Menunggu Goroutine

WaitGroup memastikan semua goroutine selesai sebelum eksekusi berikutnya dimulai.



Kontrol Proses Concurrent

Cocok untuk menjaga alur saat banyak tugas berjalan paralel di backend.



Use Case Backend

Berguna saat memanggil service, batch data, atau query independen bersamaan.


```
    "sync",  
    "time"  
}  
  
func worker(id int, wg *sync.WaitGroup) {  
    defer wg.Done()  
    fmt.Printf("Worker %d starting\n", id)  
    time.Sleep(time.Second) // Simulate work  
    fmt.Printf("Worker %d done\n", id)  
}  
  
func main() {  
    var wg sync.WaitGroup  
    for i := 1; i <= 3; i++ {  
        wg.Add(1)  
        go worker(i, &wg)  
    }  
}
```

Contoh WaitGroup

Kode ini memakai WaitGroup untuk menunggu tiga goroutine selesai agar main function tidak berakhir lebih dulu.

Channel sebagai Alat Komunikasi

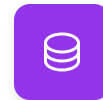
```
    "net/http"
    "fmt"
}

func helloHandler(w http.ResponseWriter, r *http.Request) {
    name := _r.URL.Query().Get("name")
    fmt.Fprintf(w, "Hello, %s!", name)
}

func main() {

    http.HandleFunc("/hello", helloHandler)
    fmt.Println("Starting server on :8080")
    http.ListenAndServe(":8080", nil)
}
```

Channel memungkinkan goroutine bertukar data, sinyal selesai, dan error dengan aman melalui komunikasi terstruktur.



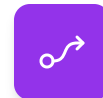
Komunikasi Aman

Channel mengirim data antar goroutine tanpa berbagi memori secara langsung.



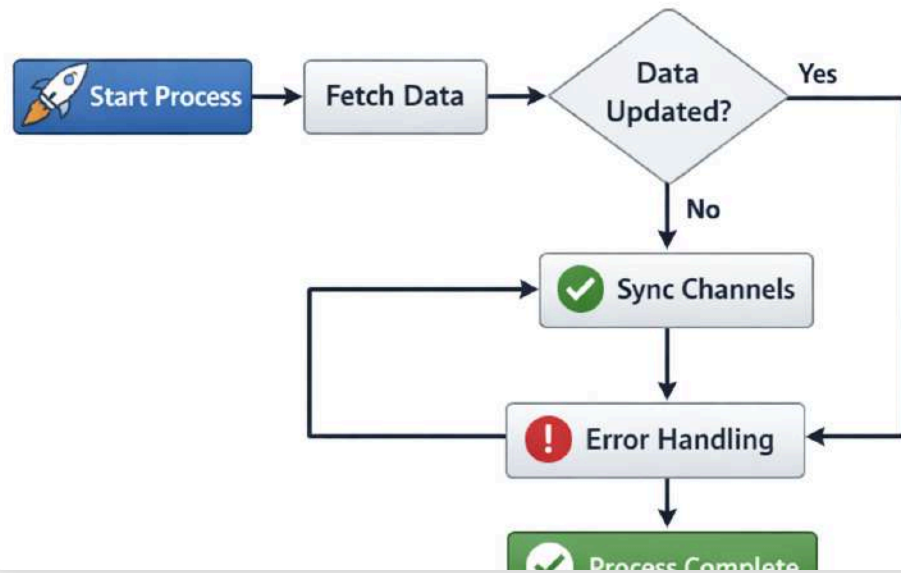
Sinkronisasi Proses

Dipakai untuk sinyal selesai, error handling, dan koordinasi antar task.



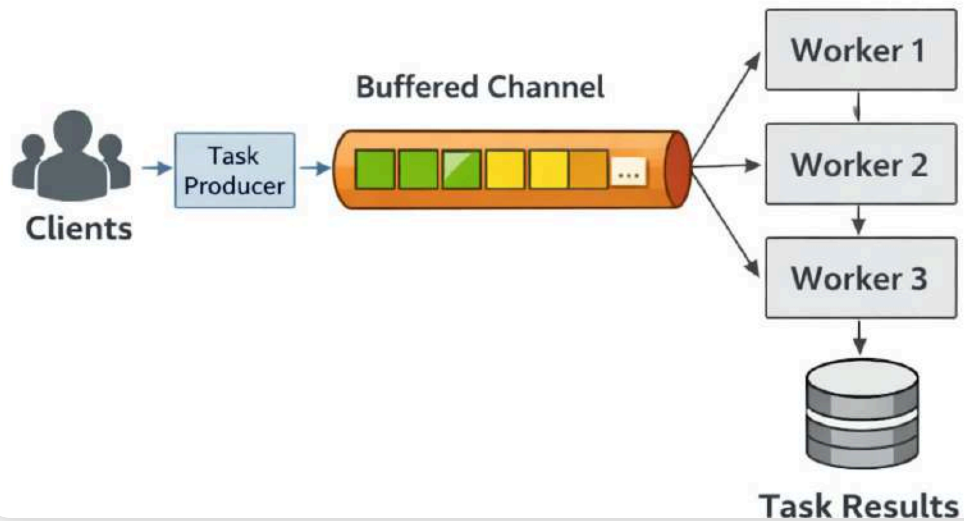
Pola Backend Umum

Cocok untuk pipeline data, worker pool, dan koordinasi pekerjaan backend.



Contoh Channel Sederhana

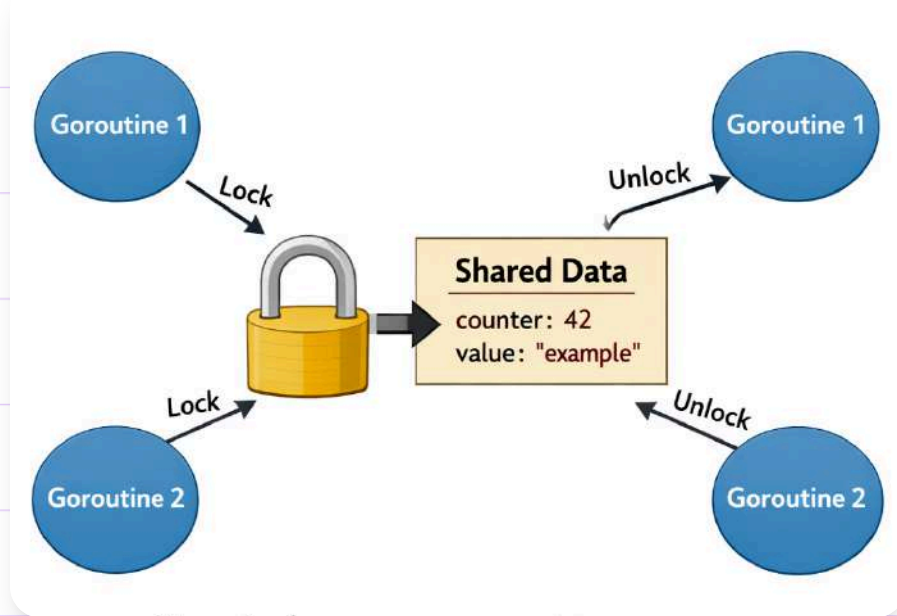
Channel dipakai untuk kirim data dan sinkronisasi goroutine.



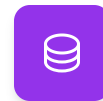
Buffered Channel dan Use Case

Buffered channel menampung data sementara untuk antrean backend.

Race Condition dan Mutex

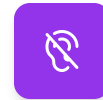


Race condition muncul saat beberapa goroutine mengubah data bersama tanpa pengamanan, sehingga hasil bisa tidak konsisten.



Race condition

Terjadi saat banyak goroutine membaca dan menulis data yang sama secara bersamaan.



Peran mutex

Mutex dari package sync mengunci akses ke shared resource agar perubahan berlangsung aman.



Contoh backend

Umum dipakai pada cache in-memory, counter, dan struktur data global di backend.

```
var counter int = 0
var mu sync.Mutex

func increment() {
    mu.Lock()
    defer mu.Unlock()
    counter++
}

func printCounter() {
    mu.Lock()
    defer mu.Unlock()
    fmt.Println("Counter:", counter)
}

func main() {
    var wg sync.WaitGroup
    for i := 0; i < 10; i++ {
        wg.Add(1)
```

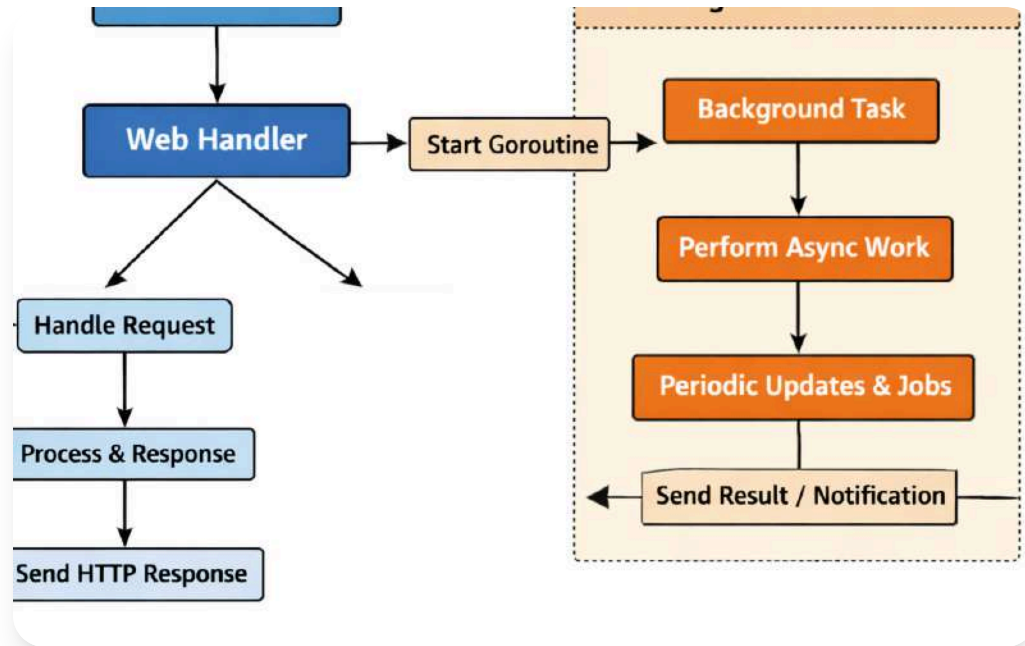
Contoh Penggunaan Mutex

Mutex memastikan hanya satu goroutine mengubah **counter** pada satu waktu, sehingga konflik akses concurrent dapat dihindari.



Concurrency dalam Framework Gin

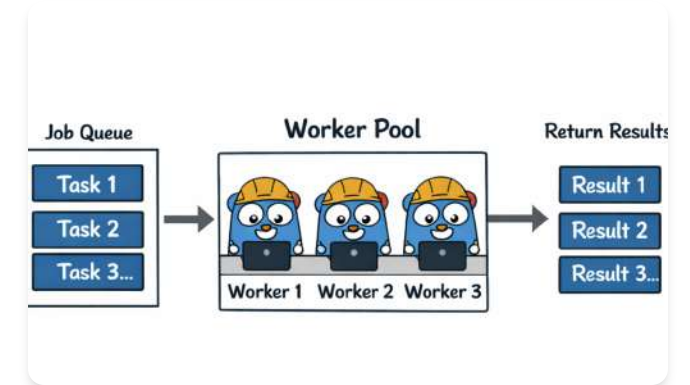
Gin efisien menangani request. Concurrency membantu tugas tambahan dengan context yang dikelola hati-hati.



Handler Gin dengan Goroutine

Handler merespons client lebih cepat, sementara proses ringan berjalan di background. Untuk tugas penting, tambahkan monitoring dan kontrol error.

Pola Worker Pool Sederhana



01 Batasi Goroutine Aktif

Worker pool mengatur jumlah goroutine yang berjalan bersamaan agar beban kerja tetap terkendali dan efisien.

02 Jaga Stabilitas Resource

Pola ini membantu mencegah penggunaan CPU dan memori berlebihan saat banyak tugas masuk secara bersamaan.

03 Cocok untuk Backend Gin

Ideal untuk upload, notifikasi, dan antrian job agar layanan tetap stabil ketika trafik meningkat.

Praktik Terbaik Concurrency



Gunakan saat perlu

Buat goroutine hanya jika memberi manfaat nyata dan hindari pemakaian tanpa kontrol.



Kelola risiko operasional

Perhatikan error handling, timeout, dan pembatasan resource agar backend tetap aman dan responsif.



Sinkronisasi yang tepat

Pakai WaitGroup untuk sinkronisasi, channel untuk komunikasi, dan mutex untuk shared state.



Latihan Praktik 1

```
func fetchAPI(url string, wg *sync.WaitGroup) {
    defer wg.Done()
    resp, err := http.Get(url)
    if err != nil {
        fmt.Println("Error:", err)
    }
    return
    fmt.Println("Fetched:", url)
    defer resp.Body.Close()
}

func main() {
    var wg sync.WaitGroup
    urls := []string{
        "https://api.example.com/data1",
        "https://api.example.com/data2",
        "https://api.example.com/data3"
    }
    for _, url := range urls {
        wg.Add(1)
    }
}
```

01 Endpoint /aggregate

Buat endpoint Gin **/aggregate** untuk mengambil data dari 3 sumber simulasi dan menggabungkannya ke satu response JSON.

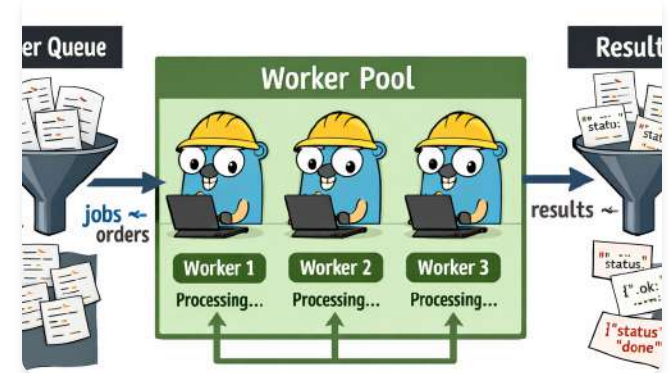
03 Bandingkan hasil

Ukur dan bandingkan waktu eksekusi **concurrent** versus **sequential** untuk melihat peningkatan performa.

02 Concurrent dengan Go

Gunakan goroutines dan WaitGroup, lalu tambahkan delay berbeda pada tiap sumber untuk mensimulasikan waktu respons.

Latihan Praktik 2



01 Endpoint Pemrosesan Order

Buat endpoint Gin `/orders/process` yang menerima beberapa order dalam bentuk array JSON.

03 Output dan Ringkasan

Setiap worker mencetak ID order yang diproses, lalu response mengembalikan jumlah order yang berhasil diproses.

02 Worker Pool Concurrent

Gunakan *buffered channel* dan 3 worker goroutine untuk memproses setiap order secara concurrent.

Penutup



"Pada pertemuan 22, concurrency di Go dipelajari melalui goroutines, WaitGroup, channel, mutex, dan penerapannya di backend Gin."

Ringkasan Materi